

Memory, Stability, and Chaos in Reservoir Computing

AMATH 383 Final Project

Taeyun Kim

Abstract

Reservoir computing processes sequential data by projecting inputs into a high-dimensional recurrent network and training only a linear readout, but its usefulness depends on a trade-off between forgetting inputs too quickly and falling into chaos. We model a leaky-integrator echo state network as a continuous-time system of nonlinear ODEs and apply linear stability analysis and bifurcation theory to locate the boundary between these regimes. Linearizing about the rest state shows that it loses stability when the spectral radius of the recurrent matrix reaches the leakage rate, recovering the classical echo state condition, and a reduced model exhibits a supercritical pitchfork. A numerical study confirms the boundary and finds that Lorenz prediction is best within the stable regime, with no advantage at the edge of chaos.

Problem Description

Processing continuous, time-varying information is a challenge in both computational neuroscience and quantitative finance. Traditional mathematical models often struggle to balance high-dimensional feature representation with computational tractability. Reservoir Computing addresses this by projecting low-dimensional sequential inputs into a high-dimensional dynamical space through a fixed recurrent network, training only a linear output layer.

An Echo State Network (ESN) is the canonical realization of this idea (Jaeger, 2001). It consists of a large, randomly connected recurrent network of nonlinear nodes, called the reservoir, whose internal weights are fixed at initialization and never trained. An input signal drives the reservoir, and its resulting high-dimensional state serves as a rich set of temporal features. Only a linear output layer, the readout, is fitted to the target. Because

the recurrent weights remain fixed, an ESN avoids the high training cost and vanishing gradients that affect conventional recurrent neural networks, while still retaining a memory of past inputs through its internal dynamics.

However, a fundamental trade-off governs any reservoir’s usefulness. When the internal connection weights are too weak, the system becomes overly contractive: perturbations decay too rapidly, past inputs are forgotten immediately, and the reservoir fails to retain useful temporal structure. When the weights are too strong, the system bifurcates into deterministic chaos, diverging exponentially from nearby initial conditions and rendering the reservoir state unpredictable and untrainable.

This project characterizes the mathematical boundary between these two regimes, called the “edge of chaos”, using local stability analysis of the reservoir’s governing ODE and bifurcation theory. We evaluate how this dynamical transition affects the network’s capacity for chaotic time-series prediction.

The most useful regime is believed to lie near this boundary. In computational neuroscience, the criticality hypothesis proposes that biological brains optimize information processing by operating near such a phase transition (te Vrugt, 2024). In quantitative finance, reservoirs tuned near the boundary may better capture the nonlinear temporal structure of volatile market data, owing to their theoretically higher memory capacity in this regime. The benefit of operating near the edge of chaos is itself debated, which makes a careful mathematical characterization of the boundary worthwhile.

Specifically, this project addresses three questions. First, where does the boundary between stable fading memory and chaotic dynamics lie for the continuous-time reservoir model? Second, how does the spectral radius of the recurrent weight matrix control the location of this boundary? Third, does operating near the boundary measurably improve performance on a chaotic time-series prediction task?

Simplifications

We adopt the standard Reservoir Computing assumptions. The recurrent weight matrix W is drawn randomly from a uniform or Gaussian distribution and held fixed for all time, and only the linear readout is trained. This is the defining simplification of the reservoir paradigm and is standard in the literature (Jaeger, 2001; te Vrugt, 2024). It replaces the difficult problem of training a full recurrent network with a tractable linear regression, at the cost of relinquishing control over the precise internal dynamics.

We take the activation function f to be the hyperbolic tangent. The function $\tanh$ is bounded and odd, which keeps the reservoir state bounded and guarantees that the origin is a fixed point of the unforced system. This makes the equilibrium analysis in the following sections clean.

We pass to continuous time. The original ESN update is a discrete map, but we take the small-step limit $\delta t \rightarrow 0$ to obtain a system of ODEs. This allows us to apply the linear stability and bifurcation tools, and it aligns the model with the continuous-time dynamics of biological neural systems. The limitation is that practical networks run in discrete time, so the continuous model approximates the implemented system.

For the stability analysis we restrict to the autonomous case, setting the input $u(t) = 0$ and the bias $b = 0$. The origin $x^* = 0$ is an equilibrium and we can linearize about it. We study this unforced system because its contraction or expansion is what determines whether past inputs fade or amplify. The limitation is that the prediction task is input-driven and non-autonomous. The Echo State Property for driven systems is governed by a related condition: $\rho(W) < 1$ is sufficient but not strictly necessary, and as Jaeger (2001) notes, a reservoir with $\rho(W) > 1$ can still possess the property when the input does not contain the null signal. We also assume the input streams $u(t)$ are bounded, which is a standard condition.

Finally, our stability results are local. Linearization characterizes the dynamics only in a neighborhood of the zero equilibrium, so the global stability of the full nonlinear system

is not resolved in this project.

Mathematical Model

While echo state networks are usually implemented as discrete-time maps, biological neural systems evolve in continuous time. We treat the reservoir as a continuous-time dynamical system: a set of N coupled, nonlinear ordinary differential equations whose state is driven by an external input. The model is not built from a conservation law. Instead, it is a state-evolution equation in which the rate of change of each node balances a linear leak against a nonlinear, input-driven update. We follow the leaky-integrator formulation of Jaeger (2001). The discrete-time update for a leaky reservoir state $x(n) \in \mathbb{R}^N$ at step n is

$$x(n+1) = (1 - \alpha \delta t) x(n) + \delta t f(W_{in} u(n+1) + W x(n) + b) \quad (1)$$

where δt is a small time step and $\alpha > 0$ is the leakage rate, which sets how quickly each node's state decays. The factor $(1 - \alpha \delta t)$ is the fraction of the current state retained at each step.

To recover a continuous-time model suitable for ODE analysis, we subtract $x(n)$ from both sides and divide by δt :

$$\frac{x(n+1) - x(n)}{\delta t} = -\alpha x(n) + f(W_{in} u(n+1) + W x(n) + b) \quad (2)$$

Taking the limit $\delta t \rightarrow 0$, the difference quotient on the left becomes a derivative, yielding a non-autonomous system of nonlinear ODEs in which the external input $u(t)$ drives the system from outside:

$$\frac{dx}{dt} = -\alpha x(t) + f(W_{in} u(t) + W x(t) + b) \quad (3)$$

The variables and parameters are defined as follows:

- $x(t) \in \mathbb{R}^N$: the high-dimensional internal state vector of the reservoir.
- $u(t) \in \mathbb{R}^M$: the time-dependent external input signal.
- $W_{in} \in \mathbb{R}^{N \times M}$ and $W \in \mathbb{R}^{N \times N}$: the fixed, randomly generated input and recurrent weight matrices.
- $b \in \mathbb{R}^N$: a fixed bias vector.
- $\alpha > 0$: the leakage rate, governing the decay of node states over time.
- $f(\cdot)$: a bounded, element-wise nonlinear activation function, chosen here as the hyperbolic tangent ($\tanh$) for its boundedness and symmetry about the origin.

To complete the model, the reservoir state $x(t)$ is mapped to an output signal by a linear readout layer:

$$y(t) = W_{out} x(t) \quad (4)$$

where $y(t) \in \mathbb{R}^K$ is the K -dimensional predicted output and $W_{out} \in \mathbb{R}^{K \times N}$ is the trained output weight matrix. Consistent with the reservoir computing paradigm, W_{in} , W , and b are fixed at initialization, so Equation (3) defines a fixed, untrained nonlinear projection of the input history into the reservoir state space. The readout weights W_{out} in Equation (4) are the only parameters fitted during training, obtained by ridge regression on the collected reservoir states to minimize the regularized squared error against the target signal. Because the readout is a static linear map that does not feed back into the reservoir, it does not affect the internal dynamics, and the stability analysis concerns only Equation (3).

Equation (3) remains the governing equation for the internal dynamics. In the autonomous case, the input and bias vanish and the system reduces to an autonomous nonlinear ODE system. This is what permits the linear stability analysis carried out in the next section, where we identify an equilibrium, linearize about it, and read off stability from the eigenvalues of the resulting Jacobian.

Solution of the Mathematical Problem

We analyze the autonomous reservoir using linear stability analysis, the standard procedure for ODE systems, applied here in its N -dimensional form. In the autonomous case $u(t) = 0$ and $b = 0$, and the governing equation (3) becomes

$$\frac{dx}{dt} = -\alpha x + f(Wx) \quad (5)$$

By definition, an equilibrium x^* satisfies $dx/dt = 0$, so

$$-\alpha x^* + f(Wx^*) = 0 \quad (6)$$

Taking $x^* = 0$,

$$-\alpha \cdot 0 + f(W \cdot 0) = f(0) = \tanh(0) = 0 \quad (7)$$

which satisfies the equation. Hence the origin is an equilibrium of the unforced reservoir,

$$x^* = 0 \quad (8)$$

To determine the stability, we compute the Jacobian matrix. The i -th component of the right-hand side is

$$F_i(x) = -\alpha x_i + f\left(\sum_{k=1}^N W_{ik} x_k\right) \quad (9)$$

Differentiating with respect to x_j , the leak term $-\alpha x_i$ contributes $-\alpha$ when $i = j$ and zero otherwise, while the activation term contributes $f'((Wx)_i) W_{ij}$. Hence

$$\frac{\partial F_i}{\partial x_j} = f'((Wx)_i) W_{ij} \quad (i \neq j), \quad \frac{\partial F_i}{\partial x_i} = -\alpha + f'((Wx)_i) W_{ii} \quad (10)$$

Collecting the diagonal leak into $-\alpha I$ and the activation terms into $D(x)W$,

$$J(x) = -\alpha I + D(x)W, \quad D(x) = \text{diag}(f'((Wx)_1), \dots, f'((Wx)_N)) \quad (11)$$

By substitution of $x^* = 0$, each argument $(Wx^*)_i = 0$, and since $f'(0) = \tanh'(0) = 1 - \tanh^2(0) = 1$, we have $D(0) = I$. Then

$$A = J(0) = -\alpha I + W = \begin{pmatrix} W_{11} - \alpha & W_{12} & \cdots & W_{1N} \\ W_{21} & W_{22} - \alpha & \cdots & W_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ W_{N1} & W_{N2} & \cdots & W_{NN} - \alpha \end{pmatrix} \quad (12)$$

The eigenvalues λ of A satisfy the characteristic equation $\det(A - \lambda I) = 0$:

$$\begin{aligned} \det(A - \lambda I) &= \det(-\alpha I + W - \lambda I) \\ &= \det(W - (\lambda + \alpha)I) = 0 \end{aligned} \quad (13)$$

This holds if and only if $\lambda + \alpha$ is an eigenvalue of W . Hence, for each eigenvalue λ_W of W ,

$$\lambda = \lambda_W - \alpha \quad (14)$$

so each eigenvalue of the Jacobian is an eigenvalue of W shifted by $-\alpha$.

The equilibrium is stable when all eigenvalues have negative real parts. That means, for every eigenvalue λ_W of W ,

$$\text{Re}(\lambda) = \text{Re}(\lambda_W) - \alpha < 0 \implies \text{Re}(\lambda_W) < \alpha \quad (15)$$

We express this through the spectral radius $\rho(W) = \max_i |\lambda_{W,i}|$. Since $\text{Re}(\lambda_W) \leq |\lambda_W| \leq$

$\rho(W)$, a sufficient condition for stability is

$$\rho(W) < \alpha \quad (16)$$

For a random W , the eigenvalues fill a disk of radius $\rho(W)$ in the complex plane, so the largest real part is close to $\rho(W)$, and the stability boundary is

$$\rho(W) = \alpha \quad (17)$$

For the normalization $\alpha = 1$, this reduces to $\rho(W) < 1$, the classical echo state condition for fading memory (Jaeger, 2001).

The result above analyzes the rest state $x^* = 0$, but in general this is not the only equilibrium. Below the stability boundary it is the unique equilibrium, while above it additional equilibria appear. To exhibit them explicitly, we analyze a reduced version of the model, a single self-coupled node with recurrent weight $w > 0$

$$\frac{dx}{dt} = -\alpha x + \tanh(wx) \quad (18)$$

Equilibria satisfy

$$\alpha x^* = \tanh(wx^*) \quad (19)$$

The origin $x^* = 0$ is always a solution. A nonzero solution exists where the line αx meets the curve $\tanh(wx)$ away from the origin, which occurs only when the curve is steeper than the line at the origin, that is when $w > \alpha$. In that case two symmetric equilibria $\pm x^*$ appear; for $w < \alpha$ the origin is the unique equilibrium.

For stability we use $F'(x^*)$ with $F(x) = -\alpha x + \tanh(wx)$:

$$F'(x) = -\alpha + w(1 - \tanh^2(wx)) \quad (20)$$

At the origin, $F'(0) = w - \alpha$, so $x^* = 0$ is stable for $w < \alpha$ and unstable for $w > \alpha$, which agrees with the general result. At the nonzero equilibria the curve $\tanh(wx)$ crosses the line αx from above, so its slope there is smaller than α , giving $w(1 - \tanh^2(wx^*)) < \alpha$ and hence $F'(\pm x^*) < 0$. The outer branches are therefore stable. The origin and the two outer branches exchange stability at

$$w = \alpha \tag{21}$$

a pitchfork bifurcation: as w increases through α , the stable rest state loses stability and gives rise to two new stable equilibria.

This reduced model shows that crossing the stability boundary creates new equilibria. The full reservoir is richer. The eigenvalues of a random W are complex, so the leading pair crosses the imaginary axis with nonzero imaginary part and the rest state loses stability through growing oscillations rather than a pitchfork. In both cases $x^* = 0$ destabilizes at the same boundary, $w = \alpha$ in the reduced model and $\rho(W) = \alpha$ in the high-dimensional system. Whether the dynamics beyond this boundary are periodic, quasi-periodic, or chaotic is a global, nonlinear question.

Results and Discussion

We complement the analysis with a numerical study of the model. The reservoir is built with a recurrent matrix W drawn at random and rescaled to a prescribed spectral radius $\rho(W)$, with leakage rate $\alpha = 1$ so that the predicted boundary lies at $\rho(W) = 1$. The autonomous system is integrated directly, while the driven system is advanced using the forward-Euler discretization of Equation (3), and the readout is trained by ridge regression. In every experiment $\rho(W)$ is the control parameter.

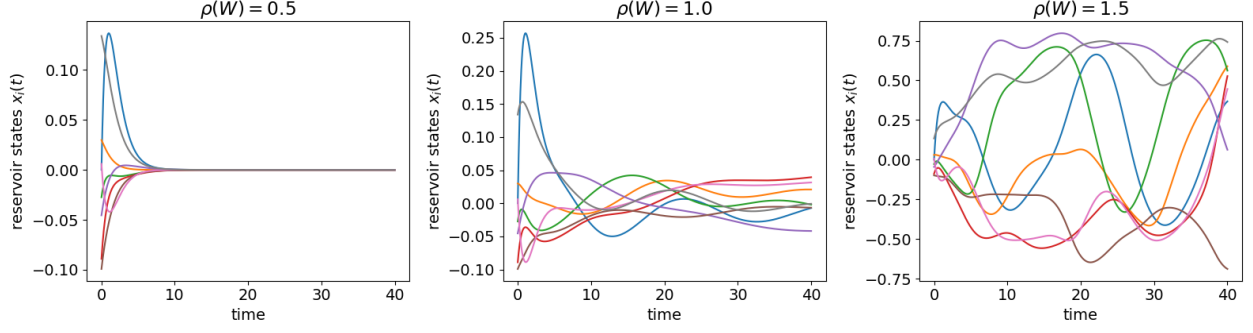


Figure 1: Autonomous reservoir states $x_i(t)$ evolving from a small perturbation at three spectral radii. Below the boundary the states decay, at the boundary they settle slowly, and above it they sustain bounded oscillations.

Figure 1 shows the autonomous reservoir states evolving from a small perturbation at three spectral radii. For $\rho(W) = 0.5$, below the boundary, every state decays to the rest state, confirming the fading-memory regime in which all Jacobian eigenvalues have negative real part. For $\rho(W) = 1.0$, at the boundary, the states settle only very slowly, consistent with a leading eigenvalue whose real part is near zero. For $\rho(W) = 1.5$, above the boundary, the states do not return to zero but sustain bounded oscillations, the signature of the unstable rest state. The amplitude stays bounded through the saturation of the tanh nonlinearity, as expected.

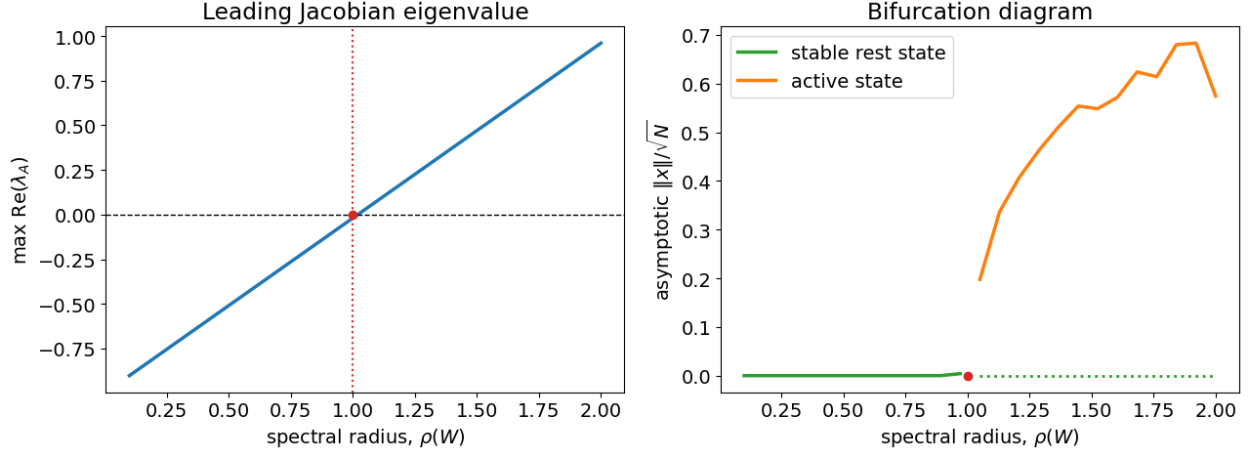


Figure 2: Left: the largest real part of the Jacobian eigenvalues against $\rho(W)$, crossing zero at $\rho(W) = \alpha$. Right: the bifurcation diagram of the asymptotic state norm, with the rest state stable below the boundary and an active state emerging above it.

Figure 2 makes the boundary quantitative. The left panel plots the largest real part of the Jacobian eigenvalues against $\rho(W)$. The analysis predicts $\lambda = \lambda_W - \alpha$ with the largest $\text{Re}(\lambda_W)$ near $\rho(W)$, so $\max \text{Re}(\lambda) \approx \rho(W) - \alpha$, a straight line crossing zero at $\rho(W) = \alpha$. The numerical curve is exactly this line, crossing zero at $\rho(W) = 1$, which is a direct confirmation of the derived boundary. The right panel is the bifurcation diagram of the asymptotic state norm. The rest state is the stable attractor below the boundary, becomes unstable above it, and a nonzero active state emerges, matching the loss of stability found analytically.

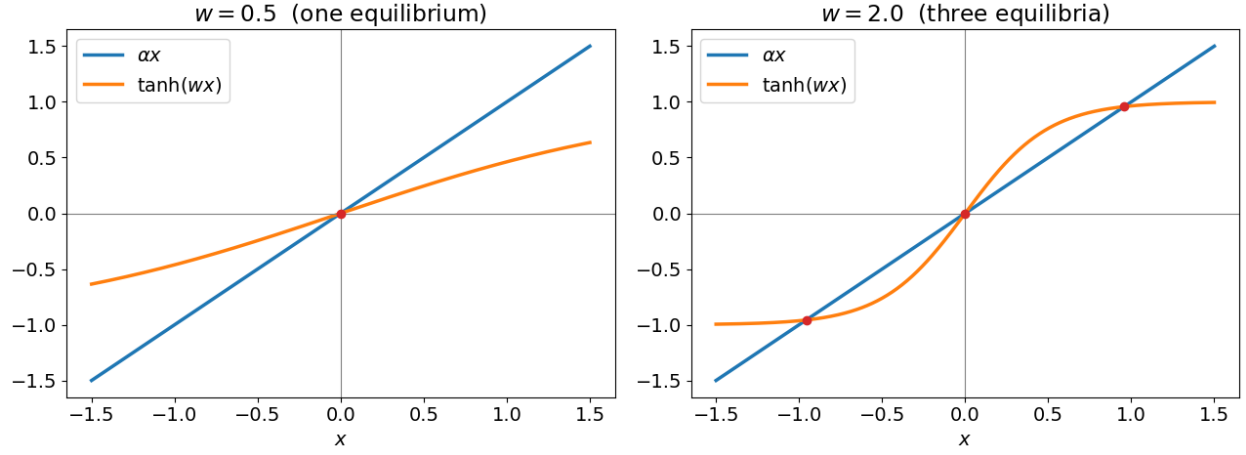


Figure 3: The line αx against the curve $\tanh(wx)$ for the reduced model. Their intersections are the equilibria: a single one for $w < \alpha$ and three for $w > \alpha$.

Figures 3 and 4 examine the reduced single-node model, which exposes the equilibrium structure that the high-dimensional norm alone cannot show. Figure 3 plots the line αx against the curve $\tanh(wx)$, whose intersections are the equilibria. For $w = 0.5$ the line is steeper at the origin and the only intersection is $x^* = 0$, and for $w = 2.0$ the curve is initially steeper and then saturates, producing the origin together with a symmetric pair $\pm x^*$.

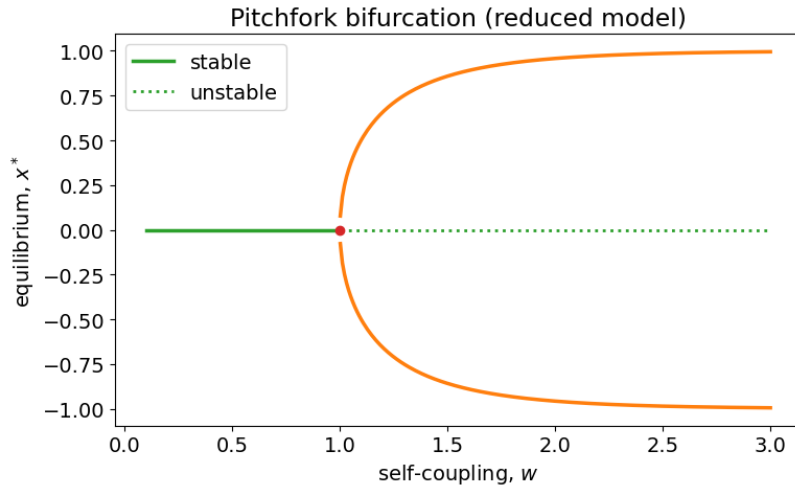


Figure 4: Equilibria of the reduced model as the self-coupling w varies. The origin is stable below $w = \alpha$ and unstable above it, while two stable branches $\pm x^*$ emerge, a pitchfork bifurcation.

Figure 4 traces these equilibria as w varies. The origin is the unique stable equilibrium below $w = \alpha$ and loses stability above it, while two stable branches $\pm x^*$ branch off. This is the supercritical pitchfork derived analytically, with the bifurcation point at $w = \alpha$ marked. The reduced model thus confirms that crossing the stability boundary is accompanied by the birth of new equilibria.

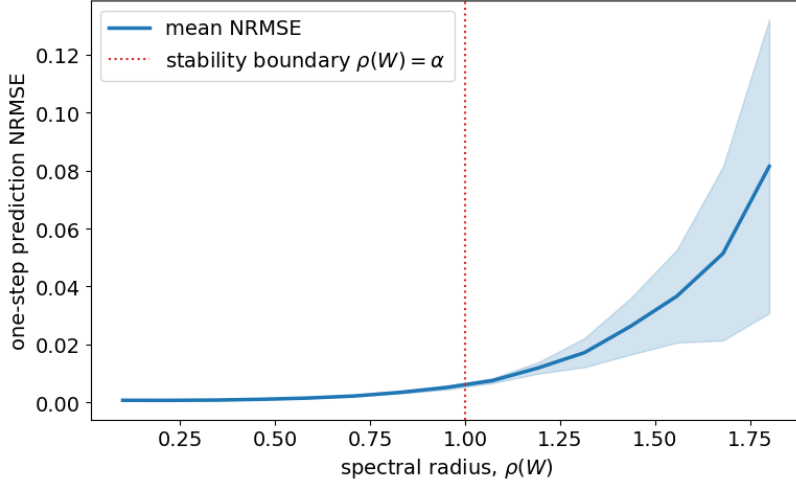


Figure 5: One-step Lorenz prediction error against $\rho(W)$, averaged over reservoir seeds, with the shaded band showing the spread. The error stays low through the stable regime and rises once $\rho(W)$ crosses the boundary.

Figure 5 reports the Lorenz prediction benchmark (Lorenz, 1963). The one-step prediction error, averaged over reservoir seeds, remains low and nearly flat throughout the stable regime and rises sharply once $\rho(W)$ crosses the boundary into instability. Performance is therefore best within the stable, fading-memory regime and degrades as the reservoir approaches and passes the chaotic boundary. We observe no improvement at the edge of chaos. The error increases monotonically through $\rho(W) = 1$ rather than dipping near it.

Taken together, these results answer the questions posed at the outset. The boundary between fading memory and instability lies at $\rho(W) = \alpha$, exactly as derived, and it is governed by the spectral radius of the recurrent matrix. On whether operating near this boundary improves prediction, our experiments do not support the edge-of-chaos hypothesis

for the Lorenz task: the best performance occurs comfortably inside the stable regime, and crossing the boundary degrades it. This is consistent with the skeptical view in the literature, where the edge-of-chaos advantage has been described as neither well defined nor empirically settled (te Vrugt, 2024). One caveat is that one-step prediction of a smooth trajectory is a relatively undemanding task that does not strongly stress the reservoir’s memory, so a benefit nearer the boundary could in principle appear for tasks requiring longer memory. We did not find a robust effect of this kind, and a fuller exploration is left to future work.

Improvement

The most direct improvement to the analysis is to move beyond local linear stability. Our results show only that the rest state loses stability at $\rho(W) = \alpha$ and do not establish whether the dynamics past the boundary are truly chaotic. Estimating the largest Lyapunov exponent, the rate at which nearby trajectories separate, might settle this.

A second extension is to analyze the input-driven system directly rather than the autonomous reduction. The prediction task drives the reservoir continuously, and the Echo State Property for driven systems is set by a contraction condition on the driven map rather than by the eigenvalues of the unforced Jacobian. The condition could be investigated by testing whether two reservoirs started from different states but given the same input converge to the same trajectory.

A third extension targets the benchmark. One-step prediction of a smooth trajectory does not stress the reservoir’s memory, which may explain the absence of any advantage near the edge of chaos. Tasks that demand longer memory, such as a memory-capacity task, would test the edge-of-chaos hypothesis more fairly.

Finally, the reservoir was left deliberately simple. Tuning the input scaling, regularization, leakage rate, and reservoir size, or adopting alternatives such as sparse connectivity or the next-generation reservoir computing approach that replaces the random reservoir with explicit nonlinear features (Gauthier et al., 2021), would likely raise accuracy.

Conclusions

We modeled an echo state network as a continuous-time dynamical system and used the linear stability analysis to characterize the boundary between its useful and unusable regimes. Linearizing the autonomous reservoir about its rest state, we found that each Jacobian eigenvalue is an eigenvalue of the recurrent matrix shifted by the leakage rate, so the rest state loses stability when the spectral radius reaches $\rho(W) = \alpha$. With the standard normalization this reproduces the classical echo state condition $\rho(W) < 1$, and a reduced single-node model showed that crossing the boundary creates new equilibria through a supercritical pitchfork bifurcation.

The numerical study confirmed this. The leading Jacobian eigenvalue crosses zero at $\rho(W) = \alpha$, the rest state gives way to sustained activity beyond it, and the predicted equilibrium structure appears in the reduced model. On the Lorenz prediction task, accuracy was best within the stable regime and degraded once the boundary was crossed, with no advantage at the edge of chaos. The analysis locates the stability boundary precisely while leaving the edge-of-chaos hypothesis unsupported for this task, in keeping with the open status of that idea in the literature.

Acknowledgements

For this project, I consulted additional sources beyond those listed in the references, including Wikipedia, blog postings, and lecture notes from various classes (AMATH383, MATH208, STAT435).

I used AI assistance (Google Gemini) to help me understand the terminology and concepts introduced at the start of the project, as a lot of mathematical terms and concepts were not introduced in the course. All of the mathematical derivations in this project were done by me, without AI's generation. Additionally, I used AI assistant for debugging the code simulation and LaTeX error debugging, but the majority of the code are written by me (I avoided excessive use of AI for coding). I also used AI to revise some of the words so that

the writing sounds more professional and similar to actual research papers. Lastly, I used Grammarly to check for correct grammar throughout the paper.

Thank you for reading my project. I have learned a lot throughout the quarter, and I really enjoyed this project even though it was very challenging.

References

- Barančok, P., & Farkaš, I. (2014). Memory Capacity of Input-Driven Echo State Networks at the Edge of Chaos. *Lecture Notes in Computer Science*, 8681, 41–48.
https://doi.org/10.1007/978-3-319-11179-7_6
- Bordenave, C., & Chafaï, D. (2012). Around the circular law. *Probability Surveys*, 9, 1–89.
<https://doi.org/10.1214/11-PS183>
- Elert, G. (2023). Lyapunov exponent. *The Chaos Hypertextbook*.
<https://hypertextbook.com/chaos/lyapunov-1/>
- Gauthier, D. J., Bollt, E., Griffith, A., & Barbosa, W. A. S. (2021). Next generation reservoir computing. *Nature Communications*, 12(1). <https://doi.org/10.1038/s41467-021-25801-2>
- Hoerl, A. E., & Kennard, R. W. (1970). Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1), 55–67.
<https://doi.org/10.1080/00401706.1970.10488634>
- Horn, R. A., & Johnson, C. R. (2013). *Matrix Analysis* (2nd ed.). Cambridge University Press.
- Hurley, L. A., & Shaheen, S. E. (2025). Reservoir computing with large valid prediction time for the Lorenz system. *ArXiv.org*. <https://arxiv.org/abs/2508.06730>
- Jaeger, H. (2001). The "echo state" approach to analysing and training recurrent neural networks. German National Research Center for Information Technology.
- Kuznetsov, Y. A. (2006). Andronov-Hopf bifurcation. *Scholarpedia*, 1(10), 1858.
<https://doi.org/10.4249/scholarpedia.1858>

- Lorenz, E. N. (1963). Deterministic nonperiodic flow. *Journal of the Atmospheric Sciences*, 20(2), 130–141.
- Politi, A. (2013). Lyapunov exponent. *Scholarpedia*, 8(3), 2722.
<https://doi.org/10.4249/scholarpedia.2722>
- 3Blue1Brown. (2016). Eigenvectors and eigenvalues. YouTube.
<https://www.youtube.com/watch?v=PFDu9oVAE-g>
- Strang, G. (2016). *Introduction to Linear Algebra* (5th ed.). Wellesley-Cambridge Press.
- Strogatz, S. H. (2015). *Nonlinear Dynamics and Chaos: With Applications to Physics, Biology, Chemistry, and Engineering* (2nd ed.). CRC Press.
- Tao, T. (2007). Random matrices: the circular law. What’s New.
<https://terrytao.wordpress.com/2007/08/23/random-matrices-the-circular-law/>
- te Vrugt, M. (2024). An introduction to reservoir computing.
 ArXiv.org. <https://arxiv.org/abs/2412.13212>
- Wolchover, N. (2013). In mysterious pattern, math and nature converge. *Quanta Magazine*.
<https://www.quantamagazine.org/in-mysterious-pattern-math-and-nature-converge-20130205/>
- cknd. (2021). pyESN: Echo state networks in Python.
 GitHub. <https://github.com/cknd/pyESN>
- Hill, C. (2016). The Lorenz attractor. *SciPython*.
<https://scipython.com/blog/the-lorenz-attractor>
- Sprott, J. C. (2015). Numerical calculation of largest Lyapunov exponent. Department of Physics, University of Wisconsin-Madison.
<https://sprott.physics.wisc.edu/chaos/lyapexp.htm>

Appendix - Code

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint
plt.rcParams.update({'font.size': 14})

# Reservoir construction
def build_W(N, rho, seed):
    rng = np.random.default_rng(seed)
    W = rng.normal(0, 1, (N, N))
    eigs = np.linalg.eigvals(W)
    W = W * (rho / np.max(np.abs(eigs)))
    return W

def build_Win(N, M, sigma_in, seed):
    rng = np.random.default_rng(seed + 1000)
    return rng.normal(0, sigma_in, (N, M))

alpha = 1.0
seed = 7

# Autonomous trajectories
def F(X, t, alpha, W):
    return -alpha * X + np.tanh(W @ X)

N = 100
```

```

TF = 40
tspan = np.linspace(0, TF, 1000)

rng = np.random.default_rng(seed)
X0 = 0.1 * rng.normal(0, 1, N)

rho_list = [0.5, 1.0, 1.5]
fig, axs = plt.subplots(1, 3, figsize=(16, 4.5))

for j, rho in enumerate(rho_list):
    W = build_W(N, rho, seed)
    sol = odeint(F, X0, tspan, args=(alpha, W))
    for i in range(8):
        axs[j].plot(tspan, sol[:, i], lw=1.5)
        axs[j].set_title(r'$\rho(W) = %.1f$' % rho)
        axs[j].set_xlabel('time')
        axs[j].set_ylabel(r'reservoir states - $x_i(t)$')

plt.tight_layout()
plt.show()

# Stability and Bifurcation
N = 100
rho_span = np.linspace(0.1, 2.0, 25)

Wbase = np.random.default_rng(seed).normal(0, 1, (N, N))
base_eigmax = np.max(np.abs(np.linalg.eigvals(Wbase)))

```

```

max_re = np.zeros(len(rho_span))
final_norm = np.zeros(len(rho_span))

rng = np.random.default_rng(seed)
X0 = 0.1 * rng.normal(0, 1, N)
tspan = np.linspace(0, 60, 600)

for i, rho in enumerate(rho_span):
    W = Wbase * (rho / base_eigmax)
    A = -alpha * np.eye(N) + W
    max_re[i] = np.max(np.real(np.linalg.eigvals(A)))
    sol = odeint(F, X0, tspan, args=(alpha, W))
    final_norm[i] = np.linalg.norm(sol[-1]) / np.sqrt(N)

fig, axs = plt.subplots(1, 2, figsize=(13, 5))

axs[0].plot(rho_span, max_re, color='tab:blue', lw=2.5)
axs[0].axhline(0, color='k', lw=1, ls='—')
axs[0].axvline(alpha, color='tab:red', lw=1.5, ls=':')
axs[0].plot(alpha, 0, 'o', color='tab:red')
axs[0].set_xlabel(r'spectral-radius,  $\rho(W)$ ')
axs[0].set_ylabel(r'max- $\mathrm{Re}(\lambda_A)$ ')
axs[0].set_title('Leading-Jacobian-eigenvalue')

stable = rho_span < alpha
axs[1].plot(rho_span[stable], final_norm[stable],

```

```

color='tab:green', lw=2.5, label='stable-rest-state')
axs[1].plot(rho_span[~stable], final_norm[~stable],
color='tab:orange', lw=2.5, label='active-state')
axs[1].plot(rho_span[~stable], np.zeros(np.sum(~stable)),
color='tab:green', lw=2, ls=':')
axs[1].plot(alpha, 0, 'o', color='tab:red')
axs[1].set_xlabel(r'spectral-radius,  $\rho(W)$ ')
axs[1].set_ylabel(r'asymptotic  $\|x\|/\sqrt{N}$ ')
axs[1].set_title('Bifurcation-diagram')
axs[1].legend()

plt.tight_layout()
plt.show()

```

Lorenz Prediction

```

def lorenz_rhs(state, t, sigma, beta, rho_L):

```

```

    X, Y, Z = state
    dX = sigma * (Y - X)
    dY = X * (rho_L - Z) - Y
    dZ = X * Y - beta * Z
    return [dX, dY, dZ]

```

```

dt = 0.02
t_lorenz = np.arange(0, 250, dt)
data = odeint(lorenz_rhs, [1.0, 1.0, 1.0],
t_lorenz, args=(10.0, 8.0/3.0, 28.0))
data = data[500:]

```

```
U = data / np.std(data, axis=0)
```

```
M = 3
```

```
def run_reservoir(U, W, Win, alpha, dt, x0):
```

```
    T = U.shape[0]
```

```
    X = np.zeros((T, W.shape[0]))
```

```
    x = x0.copy()
```

```
    for n in range(T):
```

```
        x = (1 - alpha * dt) * x +
```

```
        dt * np.tanh(Win @ U[n] + W @ x)
```

```
        X[n] = x
```

```
    return X
```

```
def ridge(X, Y, lam):
```

```
    N = X.shape[1]
```

```
    return Y.T @ X @ np.linalg.inv(X.T @ X + lam * np.eye(N))
```

```
def nrmse(pred, target):
```

```
    return np.sqrt(np.mean((pred - target) ** 2) / np.var(target))
```

```
N = 300
```

```
sigma_in = 0.5
```

```
lam = 1e-6
```

```
washout = 200
```

```
n_train = 6000
```

```
rho_span = np.linspace(0.1, 1.8, 15)
```

```
seeds = [1, 2, 3, 4]
```

```

errs = np.zeros((len(seeds), len(rho_span)))
for s, sd in enumerate(seeds):
    Win = build_Win(N, M, sigma_in, sd)
    Wbase = np.random.default_rng(sd + 5).normal(0, 1, (N, N))
    base = np.max(np.abs(np.linalg.eigvals(Wbase)))
    for i, rho in enumerate(rho_span):
        W = Wbase * (rho / base)
        X = run_reservoir(U, W, Win, alpha, dt, np.zeros(N))
        Wout = ridge(X[washout:washout + n_train],
            U[washout + 1:washout + n_train + 1], lam)
        pred = X[washout + n_train:-1] @ Wout.T
        errs[s, i] = nrmse(pred, U[washout + n_train + 1:])
    print('seed %d done' % sd)

err_mean = errs.mean(axis=0)
err_std = errs.std(axis=0)

plt.figure(figsize=(8, 5))
plt.plot(rho_span, err_mean, color='tab:blue',
lw=2.5, label='mean-NRMSE')
plt.fill_between(rho_span, err_mean - err_std,
err_mean + err_std, color='tab:blue', alpha=0.2)
plt.axvline(alpha, color='tab:red', lw=1.5, ls=':',
label=r'stability boundary- $\rho(W)=\alpha$ ')
plt.xlabel(r'spectral radius, - $\rho(W)$ ')
plt.ylabel('one-step prediction-NRMSE')

```

```

plt.legend()
plt.tight_layout()
plt.show()

# Pitchfork Bifurcation
from scipy.optimize import brentq

alpha = 1.0
w_span = np.linspace(0.1, 3.0, 300)

x_zero = np.zeros(len(w_span))
x_pos = np.full(len(w_span), np.nan)
x_neg = np.full(len(w_span), np.nan)

for i, w in enumerate(w_span):
    if w > alpha:
        g = lambda x: np.tanh(w * x) - alpha * x
        xr = brentq(g, 1e-6, 1.0 / alpha)
        x_pos[i] = xr
        x_neg[i] = -xr

stable = w_span < alpha
plt.figure(figsize=(8, 5))
plt.plot(w_span[stable], x_zero[stable],
color='tab:green', lw=2.5, label='stable')
plt.plot(w_span[~stable], x_zero[~stable],
color='tab:green', lw=2, ls=':', label='unstable')

```

```

plt.plot(w_span, x_pos, color='tab:orange', lw=2.5)
plt.plot(w_span, x_neg, color='tab:orange', lw=2.5)
plt.plot(alpha, 0, 'o', color='tab:red')
plt.xlabel(r'self-coupling,  $w$ ')
plt.ylabel(r'equilibrium,  $x^*$ ')
plt.title('Pitchfork bifurcation (reduced model)')
plt.legend()
plt.tight_layout()
plt.show()

```

Equilibrium

```

from scipy.optimize import brentq

```

```

alpha = 1.0

```

```

x = np.linspace(-1.5, 1.5, 400)

```

```

fig, axs = plt.subplots(1, 2, figsize=(13, 5))

```

```

for ax, w in zip(axs, [0.5, 2.0]):
    ax.plot(x, alpha * x, color='tab:blue',
            lw=2.5, label=r' $\alpha \cdot x$ ')
    ax.plot(x, np.tanh(w * x), color='tab:orange',
            lw=2.5, label=r' $\tanh(w \cdot x)$ ')
    ax.axhline(0, color='gray', lw=0.8)
    ax.axvline(0, color='gray', lw=0.8)
    ax.plot(0, 0, 'o', color='tab:red', zorder=5)
    if w > alpha:

```

```

xr = brentq(lambda s: np.tanh(w * s) - alpha * s,
1e-6, 1.0 / alpha)
ax.plot([xr, -xr], [alpha * xr, -alpha * xr], 'o',
color='tab:red', zorder=5)
ax.set_title(r'$w$=%.1f$--(three-equilibria)' % w)
else:
    ax.set_title(r'$w$=%.1f$--(one-equilibrium)' % w)
ax.set_xlabel(r'$x$')
ax.legend()

plt.tight_layout()
plt.show()

```